Qwen

The Horror.Place Public Core: A Tier-1 Implementation Blueprint for Runtime Contracts, Invariant Enforcement, and Secure Orchestration

Engine Startup Contract and Failure Mode Specifications

The operational integrity of the Horror.Place ecosystem hinges on a clearly defined minimum operational contract, which dictates the conditions under which a game engine may launch . This contract is not a mere suggestion but a fundamental governance boundary, codified within the public core repository through a combination of machine-readable schemas, a runtime pre-flight script, and formal acceptance tests. The design philosophy prioritizes safety and verifiability over convenience, establishing a two-tiered startup protocol: a hard refusal to launch if core governance data is unavailable, and a graceful degraded state known as "Neutral Void" when only static, placeholder data is present. This approach ensures that every running instance of the engine operates within a verified and consistent contractual framework, preventing it from entering a state of undefined or un-governed behavior. The implementation is distributed across several key files within the `horror-place-core` repository, each serving a distinct purpose in enforcing this critical contract.

The foundation of the startup contract is laid directly within the `coreschemas/invariantsv1.json` file. This schema, which governs all runtime states, must be extended to include specific rules that act as `runtime_gate`s, making the startup conditions auditable and machine-checkable . Two new rule entries, `R-RUN-001` and `R-RUN-002`, are proposed to formalize the engine's startup prerequisites. Rule `R-RUN-001` is a critical enforcement point, stating that the engine must not enter its main loop if no registry entries can be validated against their respective schemas . This rule prevents the engine from operating in a "no registry" state, which would violate the foundational principle of running on validated data. Its enforcement type is set to `"runtime_gate"`, indicating that a failure to satisfy this condition constitutes a fatal error at runtime . The second rule, `R-RUN-002`, addresses the scenario where the Orchestrator is unreachable.

It mandates that if the engine runs without content fed by the Orchestrator, it must explicitly declare itself to be in "Neutral Void" mode . This ensures that any degraded experience is not mistaken for a normal gameplay session and is clearly demarcated as structurally empty. Both rules are assigned severity levels (`critical` and `high`, respectively) and are linked to automated test hooks, such as `engine-startup-check`, which will be implemented in a dedicated script . These additions transform the abstract concept of a "minimum contract" into a concrete, verifiable part of the schema itself.

To translate these schema-level rules into executable logic, a new Python script, `scripts/engine_startup_check.py`, must be created in the public core repository . This script serves as the canonical pre-flight check for any engine consuming the Horror.Place constellation. Its implementation mirrors the validation logic already present in the Continuous Integration (CI) pipeline's `schema-validation` job, ensuring consistency between build-time and run-time checks . The script's responsibilities are systematic and multi-faceted. First, it loads all essential schemas from the `coreschemas/` directory, specifically `invariantsv1.json`, `entertainmentmetricsv1.json`, and `stylecontractv1.json` . Second, it loads all NDJSON registry files from the `coreregistry/` directory, including `events.json`, `regions.json`, `styles.json`, and `personas.json`. Third, it iterates through each non-empty line in these registry files, parses it as a JSON object, resolves the `schemaref` field to its corresponding local schema, and validates the entry against that schema using the same logic as the CI job . During this process, the script computes several boolean flags: `has_regions` and `has_styles`, which track whether at least one valid entry exists in those registries; and `has_non_placeholder_entries`, which determines if any entry contains meaningful data beyond simple placeholders (e.g., entries with a `metadata.status` other than `"draft"` or tags that do not exclusively denote a placeholder) .

Based on these computed flags, the script determines the appropriate startup mode and outputs a clear status. If the loading of schemas or registries fails, or if any validation step fails, the script exits with a non-zero status code, signaling a hard failure . This corresponds to the first tier of the startup contract: no registry, no game. If `has_non_placeholder_entries` is true and all validations have passed, the script prints `startup_mode=normal`, indicating that the engine can proceed with a full feature set . Conversely, if only placeholder entries exist but all validations pass, the script prints `startup_mode=neutral-void` . This output provides a deterministic signal for the engine to adopt the Neutral Void mode. This script is not just a tool; it is a reference implementation of the startup contract, providing a concrete guide for engine authors on how to implement the same logic in their own systems .

The final piece of the puzzle is comprehensive documentation within the `README.md` file of the `horror-place-core` repository. A new section titled "Runtime Consumers / Engine Startup Contract" must be added to formally prescribe the integration requirements for any engine developer . This section will explicitly instruct developers to incorporate a call to `engine_startup_check.py` (or embed its logic) as a mandatory step before entering the main game loop. The document will detail the expected behaviors based on the script's output: a non-zero exit must be treated as a hard failure, causing the engine to refuse to launch and log the reason . An output of `startup_mode=neutral-void` must trigger the engine to load into a predefined safe state, while `startup_mode=normal` allows for standard operation . This documented contract transforms the technical steps into a formal agreement between the public core and all downstream consumers, ensuring predictable and consistent behavior across different engine implementations.

To solidify this contract, the acceptance testing suite must be expanded to cover all scenarios. The `audit/acceptance-tests.md` file should be updated with a new test case, `AT-009 Engine Startup Contract` . This test will define a series of scenarios to verify the correctness of the startup logic. The first scenario involves running the `engine_startup_check.py` script with a complete set of non-placeholder registries, expecting the output `startup_mode=normal`. The second scenario uses only the placeholder entries provided in the core repository, expecting the output `startup_mode=neutral-void`. The third, and most critical, scenario simulates a missing or corrupted registry by temporarily removing the relevant files, expecting the script to exit with a non-zero status . By formally defining these preconditions, steps, and expected results, the acceptance test provides a regression-proof baseline for the entire startup contract, ensuring that any future changes to the schemas or scripts do not inadvertently break this fundamental aspect of the system's governance .

Finally, the "Neutral Void" mode itself must be precisely defined with canonical registry entries. To achieve this, two new entries must be added to the `coreregistry/regions.json` and `coreregistry/styles.json` files. In `regions.json`, a special neutral region entry with the ID `REG-NEUTRAL-VOID` is introduced . This entry includes metadata specifying its name, status (`"neutral"`), and tags like `"void"` and `"nexus"`, making it easily identifiable by an engine . Similarly, in `styles.json`, a style entry `STY-NEUTRAL-VOID` is added, which points to a style contract designed for minimal visual and audio intensity (e.g., low-key lighting, a `silenceHeavy` mix profile) . The `engine_startup_check.py` script is then updated to recognize the presence of these specific entries as a definitive indicator of a valid registry set suitable for launching in Neutral Void mode . The `README.md` documentation further refines the contract for this mode, stipulating that engines must load `REG-NEUTRAL-VOID` as the active region,

apply STY-NEUTRAL-VOID, disable AI and dynamic scare scheduling, and use minimal geometry and sound consistent with the style contract . This completes the specification, turning the philosophical idea of a "safe state" into a concrete, actionable, and contractually mandated feature of the Horror.Place public core.

Component	File Path	Action	Description
Schema Definition	coreschemas/invariantsv1.json	Extend rules array	Add R-RUN-001 and R-RUN-002 rules as runtime_gates to codify startup conditions.
Runtime Script	scripts/engine_startup_check.py	Create new script	Implements a pre-flight check that validates schemas/registries and determines startup_mode (normal or neutral-void).
Documentation	README.md	Add "Runtime Consumers" section	Prescribes that engines must call the startup check and adhere to its output before the main loop.
Acceptance Test	audit/acceptance-tests.md	Add AT-009 test	Defines scenarios for validating normal, neutral-void, and failure states of the startup contract.
Registry Entries	coreregistry/regions.json & coreregistry/styles.json	Add REG-NEUTRAL-VOID and STY-NEUTRAL-VOID	Provides canonical placeholder entries for the Neutral Void mode.

Rivers of Blood Charter Enforcement via Schema and Audit

The enforcement of the Rivers of Blood Charter within the Horror.Place architecture is fundamentally a problem of governance, not of asset inspection. Since the public core repository adheres to a strict zero-raw-content policy, it cannot see or inspect the very assets it is meant to govern . Instead, enforcement is achieved through a sophisticated, multi-layered system built entirely on abstract, quantitative, and auditable contracts. This system relies on three pillars: schema-defined numeric bounds, policy-driven configuration validation, and cryptographically traceable telemetry logs. The public core acts as the central authority for the first two layers—defining the immutable rules and the structure of the evidence—while the enforcement logic and classification happen in private tiers. This design ensures that the Charter's prohibitions are machine-enforceable without compromising proprietary design IP or violating the project's core architectural principles.

The primary mechanism for enforcement is the extension of the coreschemas/invariantsv1.json file with a dedicated group of rules representing the Charter's

core constraints. These rules do not contain narrative text but are precise, machine-checkable directives. For example, a rule `R-ROB-001` could be defined to enforce the absolute prohibition of sexual violence by setting a hard cap on the Sexual Violence Index (SVI) . The rule's definition would include a high severity and an enforcement type of `"schemavalidation"` for configuration files and `"runtimegate"` for engine execution . This means that any attempt to configure a system with an SVI greater than zero would fail both at build time (during CI) and at runtime. Another rule, `R-ROB-002`, would govern the explicitness ceiling (E_{max}), ensuring that no configuration can relax the boundaries set forth in the Charter's policy documents . Finally, `R-ROB-003` could encode a runtime constraint requiring a cooldown period after any event classified with a high Gore Explicitness Index (GXI), preventing an uncontrolled escalation of graphic content . These rules anchor the Charter in a quantifiable framework, shifting the burden of enforcement from qualitative judgment to mathematical validation.

To house the more complex parameters of the Charter itself, a separate, shared policy schema is introduced in the `HorrorPlace-Constellation-Contracts` repository. This schema, tentatively named `schemas/policy/rivers-of-blood-charter.v1.json`, defines the specific numeric limits and formulas that govern content boundaries . It would include fields for maximum allowed values of SVI, HVI (Hate Violence), GXI, and NSI (Nudity/Sexuality), potentially defined per content tier (e.g., standard, mature, research) . It would also define the coefficients (C, R, H) used to calculate the dynamic explicitness ceiling (E_{max}). Critically, this schema is kept GitHub-safe because it contains only numbers, IDs, and formula parameters, not raw content . The public core's `invariantsv1.json` can then reference this policy schema, creating a dependency that ensures any change to the Charter's underlying parameters is tracked and managed. This separation of concerns is vital: the core defines the *interface* and *constraints*, while the Constellation-Contracts repository manages the evolving *policy*.

The third pillar of enforcement is a standardized, auditable telemetry schema for logging content boundary decisions. This schema, `schemas/telemetry/content-boundary-judgment-v1.json`, is defined in the Constellation-Contracts repository and serves as the canonical format for all modules that make content decisions . An NDJSON event conforming to this schema is generated whenever a classifier or policy engine evaluates a piece of content. The payload of this event is rich with metadata but devoid of any raw content. It would include fields such as the `sessionId`, the timestamp of the decision, the computed content indices (SVI, HVI, GXI, NSI), the calculated explicitness level (E) and its theoretical maximum (E_{max}), the final decision (`"allow"`, `"soften"`, `"refuse"`, `"imply"`), and a reference to the specific policy (`policyRef`) that was used to make the call . This creates an immutable, cryptographic trail that links a runtime decision back to the governing Charter parameters, allowing for

deep audits of engine behavior without ever needing to inspect the prohibited assets themselves. The public core's CI and audit pipelines can be configured to validate that all outgoing telemetry conforms to this schema.

To ensure these mechanisms are rigorously applied, the audit plan and CI pipelines must be enhanced. The `audit/api-audit-plan.md` file needs to be updated with new tests specifically targeting Rivers of Blood compliance. Test `AUD-ROB-001` would verify that the `invariantsv1.json` schema contains the required `R-ROB-*` rules with the correct enforcement types and severities . Test `AUD-ROB-002` would perform a deeper scan of all relevant schemas in both the `horror-place-core` and `HorrorPlace-Constellation-Contracts` repositories to ensure that no public-facing configuration allows an SVI greater than zero, effectively checking the "hard zero" mandate . Test `AUD-ROB-003` would validate that any explicitness policy schema keeps its coefficients within the pre-agreed governance ranges, preventing configurations from weakening the Charter's intent . These audit tests are automated and integrated into the CI process, acting as a constant guardian against regressions in the governance framework.

Finally, the entire concept is bound together by a formal acceptance test. The `audit/acceptance-tests.md` file should include `AT-010 Rivers of Blood Invariants`, which provides a high-level verification of the setup . The precondition for this test is the existence of `coreschemas/invariantsv1.json`. The steps involve parsing this file and asserting the presence of all required `R-ROB` rules with their correct fields and enforcement strategies. The expected result is that the assertion passes, confirming that the core repository is correctly configured to uphold the Rivers of Blood Charter. This acceptance test serves as a living document, guaranteeing that the architectural vision for Charter enforcement remains intact across all development iterations. By combining schema-level constraints, policy-driven configuration, auditable telemetry, and rigorous CI/CD validation, the Horror.Place public core successfully translates a sensitive ethical charter into a robust, verifiable, and secure technical framework.

Component	Repository	File Path	Action	Description
Invariants Rules	`horror-place-core`	`coreschemas/invariantsv1.json`	Add `R-ROB-*` rules	Define schema and runtime constraints for SVI, GXI, and E_max.
Policy Schema	`HorrorPlace-Constellation-Contracts`	`schemas/policy/rivers-of-blood-charter.v1.json`	Create new schema	Defines numeric caps and E_{max} coefficients for the Charter.
Telemetry Schema	`HorrorPlace-Constellation-Contracts`	`schemas/telemetry/content-boundary-judgment-v1.json`	Create new schema	Specifies the NDJSON structure for auditing content boundary decisions.
Audit Plan Updates	`horror-place-core`	`audit/api-audit-plan.md`	Add `AUD-ROB-*` tests	Creates automated checks to scan schemas and verify Charter compliance.
Acceptance Test	`horror-place-core`	`audit/acceptance-tests.md`	Add `AT-010` test	High-level test to verify the presence and validity of Rivers of Blood invariants.

Surprise Director Stubbing and Telemetry Integration

The algorithmic heart of the Horror.Place experience, the "Surprise Director," represents a significant intellectual property asset. To protect its proprietary heuristics—such as scare cadence patterns, trigger thresholds, and randomization structures—from being reverse-engineered by players or competitors, its implementation is deliberately excluded from the public core repository . Instead, the public core establishes a strict, declarative contract that downstream, private repositories must fulfill. This contract consists of three parts: a Lua API stub that defines the calling convention, a schema for director instructions, and a schema for director decision telemetry. The public core ships only the stubs and schemas, forcing engine implementations to vendor in the real logic from trusted private sources. This design perfectly aligns with the project's philosophy of separating public contracts from private implementation, ensuring that the fear kernel remains unpredictable and proprietary.

The cornerstone of this contract is the creation of a new Lua module in the public API stub directory. The file `public-api/hdirector.lua` is created to expose functions under a `H.Director` namespace . However, unlike other stubs which might contain minimal logic, the `requestBeat` function in the public core version is a pure placeholder. Its sole purpose is to inform the consumer that the implementation is external. The function signature might be `H.Director.requestBeat(sessionId, context)`, where `context` is a table containing information like `regionId`, `tileId`,

and a snapshot of entertainment metrics . Inside the public core, this function must return a specific error envelope: `{ ok = false, data = nil, error = { code = "DIRECTOR_NOT_IMPLEMENTED", message = "Director implementation is provided by private vaults." } }` . This makes it impossible to build a functional engine from the public core alone, as any call to request a surprise beat would immediately fail. The `logDecision` function serves a similar purpose, acting as a no-op that accepts a `decisionEnvelope` but performs no actual logging, thereby defining the shape of the required telemetry without implementing it . This deliberate brokenness is the key to security; it forces the build system to resolve the dependency from a private source.

Complementing the Lua stubs are two new JSON schemas that define the structure of the data exchanged with the director. The first, `coreschemas/director-instructionv1.json`, specifies the format of a request sent *to* the director . This schema is purely structural, defining the shape of the command rather than the strategy for issuing it. It would include a `$id` (e.g., `schema.Horror.Place.core.schemas.director-instructionv1`), a `type` of `object`, and a `required` array listing all mandatory fields. Key properties would include `instructionId` (a UUID), `sessionId` (a UUID), `timestamp` (ISO date-time), `regionId` and `tileId` (opaque strings), `requestType` (an enum like `["surprise", "relief", "foreshadow"]`), `intensity` (a number between 0 and 1), and optional fields like `cooldownUntil` (a timestamp) and `tags` (an array of strings for categorization) . This schema ensures that any instruction received by the private director implementation has a consistent, validated structure.

The second schema, `coreschemas/director-decision-eventv1.json`, defines the structure of the log entry produced *by* the director after a decision has been made . This is crucial for auditing and telemetry. The schema would define an `eventId`, `sessionId`, and `timestamp`. It would contain an embedded `metricsSnapshot` object, capturing the state of UEC, EMD, CDL, ARR, and DET at the moment of the decision. It would also include `requested` and `granted` objects, both of which must conform to the `director-instructionv1.json` schema, allowing for comparison between what was asked for and what was actually executed (which might be "softened" due to budget constraints). The `decision` field would be an enum (`"executed"`, `"skipped"`, `"queued"`, etc.), and `policyRefs` would hold a list of references to the specific policies (like Rivers of Blood) that influenced the outcome . By mandating this structure, the public core ensures that all director-related events are logged in a standardized, machine-readable format suitable for analysis and audit trails, without revealing the underlying logic that produced them.

The final and most critical wiring action is configuring the build system of downstream repositories, such as `HorrorPlace-Codebase-of-Death`, to correctly consume this contract . The private director implementation, for example a file `runtime/director/surprisedirector.lua`, must contain the real behavioral logic. The build process must be designed to override the public stub. This is typically achieved by controlling the module search path, ensuring that the private director module is found and loaded before the public one during compilation or linking. Alternatively, dependency injection frameworks can be used to inject the private implementation into the engine's dependency graph, replacing the default public stub. This step is the practical enforcement of the contract; without it, the engine would remain non-functional. The public core's role ends at providing the interface; the responsibility of fulfilling that interface falls to the private tiers.

For maximum security, an additional CI guardrail can be implemented. A custom job in the `HorrorPlace-Constellation-Contracts` repository or a separate tooling repository can be created to scan the public API stubs in `horror-place-core/public-api/` . This job would specifically look for files named `<em>director</em>.lua`. If such a file is found, the job would assert that it contains only the minimal stub implementation described above. It could check the file's length, the number of executable lines, or the absence of certain keywords or internal module references . If the stub begins to grow and contain real logic, the CI job would fail, sending a clear signal to developers that they are violating the architectural contract. This automated check acts as a powerful deterrent, preserving the strict separation between public interfaces and private implementation. By combining these four elements—a broken Lua stub, two detailed schemas, build-system overrides, and optional CI guards—the public core successfully protects the director's intellectual property while providing a robust and extensible contract for engine development.

Component	File/Module	Action	Description
Lua Stub	`public-api/hdirector.lua`	Create new module	Exposes `H.Director.requestBeat()` and `H.Director.logDecision()` functions. In the public core, `requestBeat` returns a `DIRECTOR_NOT_IMPLEMENTED` error.
Instruction Schema	`coreschemas/director-instructionv1.json`	Create new schema	Defines the structure of a director instruction, including `requestType`, `intensity`, `regionId`, and other metadata.
Decision Schema	`coreschemas/director-decision-eventv1.json`	Create new schema	Defines the structure of a telemetry event for director decisions, including `metricsSnapshot`, `requested`, `granted`, and `decision` fields.
Build System Wiring	Downstream Repos (e.g., Codebase-of-Death)	Configure module resolution	Set up the build system to vendor in the private director implementation and override the public stub.
CI Guardrail	`HorrorPlace-Constellation-Contracts`	Add custom CI job	Scan public `hdirector.lua` for real logic; fail CI if the stub deviates from its minimal implementation.

Orchestrator Registry Consumption and State Transition Boundaries

The relationship between the Horror.Place engine and the Orchestrator service is a classic consumer-producer model, where the Orchestrator acts as the authoritative producer of the game's persistent data—its registries—and the engine is the disciplined consumer that must read this data safely and predictably . A critical aspect of this relationship is the management of state transitions. Allowing the engine to hot-reload or poll registry data (such as regions, personas, or events) at arbitrary times could lead to catastrophic failures known as "Reality Tears," where the game world becomes inconsistent and crashes [39] [42]. To prevent this, the public core repository must specify a strict integration pattern that confines registry updates to discrete, well-defined moments in the player's experience: upon respawning or crossing a region boundary. This ensures that the engine always operates on a consistent, atomic snapshot of the game world.

The Orchestrator's core responsibility is to manage the registry store, which holds the authoritative versions of all public data: events, regions, styles, and personas . It achieves this by polling private vaults for signed artifact descriptors, verifying their authenticity through cryptographic signatures and canonical hashing, and then applying updates to the registry store atomically . If a batch of descriptors is valid, the Orchestrator updates the registries and appends an `artifact_ingested` entry to its audit log. If any descriptor in the batch fails verification, the Orchestrator rolls back the entire update to the previous state of the registry store and appends an `artifact_rejected` entry . This rollback mechanism is what guarantees atomicity. From the engine's perspective, a successful update from the Orchestrator is not an incremental patch but a single, indivisible transaction that brings the entire registry set from one consistent state to another. The engine's task is to request these transactions at safe moments.

The public core's role is to communicate these safety constraints to engine developers through clear documentation and, where possible, schema-level guidance. The `README.md` file must contain a dedicated section on "Orchestrator Integration" that explicitly forbids hot-reloading registries while the player is inside a region . It must state that the engine should treat the Orchestrator as a source of snapshots, not a live stream. The recommended pattern is for the engine to poll the Orchestrator's API for updated registries only at two specific, discrete transition points in the game loop: when a player respawns, or when they cross a boundary between two distinct regions . At these moments, the engine requests a fresh snapshot of the relevant registries. Once the snapshot is received and validated, the engine can safely transition its internal state to match the new world state. This disciplined consumption pattern prevents the engine

from ever trying to reconcile conflicting data from two different versions of the registry simultaneously, which is the root cause of "Reality Tears."

While the public core does not duplicate the Orchestrator's internal logic for polling, hashing, or rollback, it can reinforce the state-transition contract through its schemas. For instance, the `RegionContract` schema could define relationships between entities. The Orchestrator's atomic updates ensure these relationships hold true post-update. The engine can then rely on these invariants when processing a new snapshot. For example, a rule in `invariantsv1.json` could state that every `Persona` listed in a `Region`'s metadata must have a corresponding entry in the `personas` registry. Because the Orchestrator's transactional updates maintain this integrity, the engine can assume that any valid registry snapshot it receives is self-consistent. This allows the engine to focus on rendering and gameplay logic without needing to perform complex reconciliation or conflict-resolution algorithms.

The public API specification, which lives in the `Horror.Place` repository, also plays a crucial role in defining this interaction pattern . The API endpoints exposed by the Orchestrator should be designed to support this pull-based, snapshot-oriented model. Endpoints like `GET /registries/{registryName}` and `GET /registries/{registryName}/{id}` allow the engine to retrieve specific registry data, while a bulk endpoint like `POST /registries/snapshot` could be used to request an atomic update of all registries at once . The API spec should document the intended usage patterns, reinforcing the guidance provided in the `README.md`. For example, it should clarify that a call to fetch a region's data should return the data as it existed at a single point in time, not a delta since the last call. By designing the API to fit the discrete state-transition model, the public core helps to prevent misuse and encourages best practices among engine developers.

In essence, the public core's contribution is to define the terms of engagement. It establishes that the Orchestrator is the ultimate source of truth for the game's persistent data and that the engine must interact with it as a passive, disciplined reader. The "wiring" is less about code and more about establishing a protocol. The engine wires itself to the Orchestrator by implementing the polling logic at the correct lifecycle hooks (respawn, region transition). The public core wires itself to the engine by documenting this protocol, providing the schemas that describe the data, and defining the API that enables the communication. This division of labor ensures that the complexity of managing a globally consistent, distributed registry is handled by the Orchestrator, while the engine remains focused on its core responsibility: delivering a terrifying experience based on a stable and reliable world state.

Aspect	Guidance for Engine Developers	Mechanism in Public Core
Polling Frequency	Poll the Orchestrator for updated registries only at player respawn or region transition. Do not hot-reload data while the player is inside a region.	Update `README.md` with "Orchestrator Integration" section detailing this pattern.
State Management	Treat registry updates from the Orchestrator as atomic, indivisible snapshots. Never mix data from two different snapshots within a single session.	Document that the Orchestrator provides atomic, idempotent updates with rollback.
API Usage	Use `GET /registries/{registryName}` and `GET /registries/{registryName}/{id}` to retrieve data for a consistent state.	Provide a public API specification that supports this pull-based, snapshot-oriented model.
Data Integrity	Trust that the Orchestrator maintains the integrity of the registry data. You can rely on invariants (e.g., valid foreign key references) holding true in any received snapshot.	Extend `invariantsv1.json` with rules that describe the properties of a valid registry state.
Error Handling	If the Orchestrator is unreachable during a permitted polling window, the engine should fall back to the last known-good registry snapshot. If no valid snapshot exists, it must refuse to launch.	Formalize this in the "Engine Startup Contract" (see Section 1), where a missing registry is a hard failure.

Cross-Repository CI Validation and Contract Synchronization

The integrity of the entire Horror.Place constellation depends on the strict synchronization of its interconnected repositories. Changes made in one repository, such as a modification to a core schema in `Horror.Place`, can have cascading effects on others, including the `Orchestrator`, `Constellation-Contracts`, and engine-specific vaults like `Codebase-of-Death`. Without a mechanism to detect and prevent such breaking changes, the system risks drifting out of alignment, leading to subtle bugs, runtime errors, and a breakdown of the trustless, contract-based architecture. To mitigate this risk, the implementation guide mandates the creation of cross-repository CI workflows. These workflows automate the process of verifying compatibility between repositories, treating them as interdependent components rather than isolated silos. They serve as the operational backbone of the governance framework, catching contract violations early in the development cycle and reducing the cognitive overhead on developers who must manually track inter-dependencies.

A concrete example of this principle in action is the creation of a new GitHub Actions workflow in the `HorrorPlace-Constellation-Contracts` repository, such as `.github/workflows/core-contract-check.yml` . This workflow is designed to act as a gatekeeper, ensuring that changes to the broader governance framework do not introduce regressions in the core repository. The workflow's logic would be

straightforward yet powerful. First, it would check out the `HorrorPlace-Constellation-Contracts` repository. Then, as a dependency, it would clone the `horror-place-core` repository into the workspace. This allows the workflow to access and validate the contents of the core repository. The subsequent steps would execute a series of validation scripts and checks against the cloned core repository.

One of the primary checks would be to run the `scripts/engine_startup_check.py` script, which was developed to enforce the engine startup contract . The workflow would capture the script's output and assert that it behaves as expected—for instance, that it recognizes the presence of the canonical Neutral Void region and style entries. This ensures that the core's data integrity is maintained. Other checks would target the schemas themselves. The workflow could invoke a schema validator to confirm that all schemas in `horror-place-core/coreschemas/` are syntactically valid according to the JSON Schema standard. More importantly, it could run custom scripts that validate the specific content of the invariants and policies. For example, a script could parse `coreschemas/invariantsv1.json` and check for the presence and correct formatting of the `R-RUN-<em>` and `R-ROB-</em>` rules, failing the workflow if they are missing or malformed . This automated check provides a strong guarantee that the core repository's foundational contracts are always in a valid and compliant state.

This pattern extends beyond just the core. The CI pipeline in `Constellation-Contracts` could be expanded to include jobs that validate the entire constellation. For example, it could have a job that clones `horror-place-core`, `horror-place-orchestrator`, and `horror-place-codebase-of-death` to perform end-to-end integration tests. While full integration testing might be too slow for every push, targeted checks are highly effective. A job could verify that the Orchestrator's descriptor-parsing logic is compatible with the latest registry formats defined in the core. Another job could ensure that the Lua stubs in the public API conform to the specifications outlined in the Constellation-Contracts documentation. These checks are not about duplicating functionality but about ensuring that the contracts defined in one place are still satisfied by the implementations in another. The goal is to create a network of dependency-aware CI jobs that collectively form a shield against breaking changes.

The implementation guide should provide skeletal but functional examples of these workflows to lower the barrier to adoption. A minimalistic GitHub Actions workflow YAML snippet would illustrate the key concepts: using `needs:` to manage job order, `uses: actions/checkout@v4` to get the source code, and `run:` commands to execute the validation scripts. For instance:

```
name: Core Contract Compliance Check

on: [push, pull_request]

jobs:
  validate-core:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Constellation Contracts
        uses: actions/checkout@v4
        with:
          repository: HorrorPlace/Constellation-Contracts
          token: ${{ secrets.CI_TOKEN }}

      - name: Checkout Horror.Place Core
        run: git clone https://github.com/Doctor0Evil/Horror.Place.git cor

      - name: Run Startup Check
        run: |
          cd core_repo
          python scripts/engine_startup_check.py
        # Assert exit code is 0

      - name: Validate Invariants Schema
        run: |
          cd core_repo
          # Run a custom script to check for R-RUN-<em> and R-ROB-</em> ru
          python scripts/check_invariants.py
        # Assert script output is positive
```

This example demonstrates how one repository can programmatically inspect another. The `CI_TOKEN` secret would be used to grant the workflow access to private repositories if necessary. By providing such templates, the public core repository gives other teams a blueprint for building their own dependency-aware CI, fostering a culture of shared responsibility for maintaining the integrity of the entire system.

Ultimately, these cross-repository CI workflows are the automation layer that brings the architectural vision to life. They translate high-level principles like "schemas are contracts" and "repositories are interdependent" into low-level, actionable checks that run automatically. They reduce the reliance on manual coordination and peer review for

catching breaking changes. By embedding these checks within the development process, the Horror.Place constellation can evolve in a controlled and predictable manner, ensuring that the intricate web of dependencies between its components remains healthy and aligned with the project's core governance goals. The public core, in turn, serves as the anchor for these checks, providing the stable, canonical schemas and scripts that all other repositories are held accountable to.

Workflow Goal	Trigger	Steps	Expected Outcome
Core Contract Validation	Push/Pull Request to `Constellation-Contracts`	1. Clone `Constellation-Contracts`. 2. Clone `horror-place-core` as a dependency. 3. Run `engine_startup_check.py`. 4. Run custom scripts to validate `invariantsv1.json` for required rules.	Fail the workflow if the core's startup contract is violated or its invariants are invalid.
Policy Schema Consistency	Push/Pull Request to `Constellation-Contracts`	1. Clone `Constellation-Contracts`. 2. Clone `horror-place-core`. 3. Validate that schemas in both repos use compatible `schemaref` prefixes and versions.	Fail if a schema in one repo references a non-existent or incompatible schema in another.
Public API Linting	Push/Pull Request to `horror-place-core`	1. Run existing `lua-lint` job on `public-api/*.lua`. 2. Add a new check to ensure no `api-leak-test` triggers occur.	Pass only if all public API stubs are syntactically correct and free of raw content leaks.
Downstream Consumer Test	Scheduled (e.g., nightly)	1. Clone `horror-place-core`. 2. Clone a representative downstream repo (e.g., `Codebase-of-Death`). 3. Attempt to build the downstream repo against the core's HEAD.	Fail if the build breaks due to a contract violation in the core. This acts as a smoke test for real-world impact.

Reference

1. Session 9: Verification and Validation - MIT OpenCourseWare https://ocw.mit.edu/courses/16-842-fundamentals-of-systems-engineering-fall-2015/resources/session-9-verification-and-validation/
2. Kali Linux Misconceptions: Separating Fact from Fiction - LinkedIn https://www.linkedin.com/posts/joas-antonio-dos-santos_stop-using-kali-linux-i-started-noticing-activity-7420090788501508096-XTTD
3. How to Write Technical Specs That Actually Ship - HackerNoon https://hackernoon.com/how-to-write-technical-specs-that-actually-ship

4. Sample datasets and evaluators - Amazon SageMaker AI https://docs.aws.amazon.com/sagemaker/latest/dg/model-customize-open-weight-samples.html
5. SpecEval: Evaluating Model Adherence to Behavior Specifications https://arxiv.org/html/2509.02464v1
6. Pro Ajax and the .NET 2.0 Platform - Springer Nature https://link.springer.com/content/pdf/10.1007/978-1-4302-0183-0.pdf
7. Programming F# 3.0 - Chris Smith | PDF - Scribd https://www.scribd.com/document/493056444/Programming-F-3-0-Chris-Smith
8. [PDF] Proceedings of the Thirteenth Workshop on Graph-Based Methods ... https://aclanthology.org/D19-53.pdf
9. KIRI Engine:3D Scanner & LiDAR - App Store - Apple https://apps.apple.com/us/app/kiri-engine-3d-scanner-lidar/id1577127142
10. Laura Jeffords Greenberg's Post - LinkedIn https://www.linkedin.com/posts/laurajeffordsgreenberg_claude-is-my-operating-system-its-a-clm-activity-7440014004963467264-w2Lz
11. Autonomous Agent Orchestration Layer | Priyen Pillay posted on the ... https://www.linkedin.com/posts/priyen-pillay_its-arc-again-priyens-agent-and-apparently-activity-7442458563874156545-c8rf
12. Surviving continuous deployment in distributed systems https://www.thoughtworks.com/insights/blog/continuous-delivery/surviving-continuous-deployment-distributed-systems
13. [PDF] D5.3 - IaC Execution Manager - v3 - European Commission https://ec.europa.eu/research/participants/documents/downloadPublic?documentIds=080166e5fc788f54&appId=PPGMS
14. Performance Improvements in .NET 9 - Microsoft Developer Blogs https://devblogs.microsoft.com/dotnet/performance-improvements-in-net-9/
15. Changes - curl https://curl.se/changes.html
16. AI Agents for Knowledge Work: Claude Code vs Cowork - LinkedIn https://www.linkedin.com/posts/vha14_satya-nadella-bg2-w-bill-gurley-brad-activity-7420580238205640704-M38_
17. New short course: LLMs as Operating Systems: Agent Memory ... https://www.linkedin.com/posts/andrewyng_new-short-course-llms-as-operating-systems-activity-7260353293229436928-XR6w
18. detecting real-world AI scheming incidents with open-source ... - arXiv https://arxiv.org/html/2604.09104
19. Validating JSON against Swagger API schema - Stack Overflow https://stackoverflow.com/questions/39275947/validating-json-against-swagger-api-schema

20. [PDF] Deploying and Installing SUSE AI https://documentation.suse.com/suse-ai/1.0/pdf/AI-deployment_en.pdf
21. A Survey of Large Language Models - arXiv https://arxiv.org/html/2303.18223v16
22. MVP Communities https://mvp.microsoft.com/
23. Anshuman Kundu - ASDE-2 @ Publicis Sapient - LinkedIn https://in.linkedin.com/in/anshuman-kundu-7a481b213
24. The 'Access-Control-Allow-Origin' Header Contains Multiple Values ... https://stackoverflow.com/questions/59668753/the-access-control-allow-origin-header-contains-multiple-values-however-i-onl
25. Debian -- Prospective packages https://www.debian.org/devel/wnpp/prospective
26. Damascus Station - Unknown | PDF - Scribd https://www.scribd.com/document/695666302/Damascus-Station-Unknown
27. Responsive Web | PDF | Windows 10 | Mobile App - Scribd https://www.scribd.com/document/358568553/Responsive-Web
28. [PDF] ROMA: Recursive Open Meta-Agent Framework for Long-Horizon ... https://arxiv.org/pdf/2602.01848
29. OpenClaw's Skill-Based AI Architecture for Maintainable Agents https://www.linkedin.com/posts/jatingargiitk_theres-something-deep-about-what-openclaw-activity-7432294531875762176-227j
30. Reflections on Pride Month: A love so bright, it's blinding - LA Times https://www.latimes.com/california/newsletter/2022-06-21/pride-gay-los-angeles-parenting-essential-california
31. How do you make json ref to local files? - Stack Overflow <a href="https://stackoverflow.com/questions/623 1000+/mo Saved https://www.linkedin.com/posts/bigtechmind_5-claude-vibe-coding-skills-5-tools-replaced-activity-7445854689226543105-4SbB
32. Requested packages, organized by age - Debian https://www.debian.org/devel/wnpp/requested_byage
33. https - cs.Princeton https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt
34. llms-full.txt - Namely https://namely.com/llms-full.txt
35. Download book PDF - Springer Nature https://link.springer.com/content/pdf/10.1007/978-1-4302-0810-5.pdf
36. Liz Ryan's Post - LinkedIn https://www.linkedin.com/posts/lizryan_scary-workplace-history-my-first-full-time-activity-7363653486871281664-1y8G

37. Decoding AI Complexity: Controlled Decision Loops in Coding Agents https://www.linkedin.com/posts/lakin-mohapatra_most-people-overestimate-how-complex-modern-activity-7429124907709620224-dmIs
38. Trudvang Chronicles Snowsaga | PDF - Scribd https://www.scribd.com/document/614885189/Trudvang-Chronicles-Snowsaga
39. file 954 kB https://ocw.mit.edu/courses/24-914-language-variation-and-change-spring-2019/8a6470ebb4308a0cc874f73fe24d7b61_CelexLemmasInTranscription-DISC.txt
40. dictionary.txt - CMU School of Computer Science http://www.cs.cmu.edu/~ab/15-110F10/labs/lab4/startercode/dictionary.txt
41. Scion Masks of The Mythos (Final Download) | PDF - Scribd https://www.scribd.com/document/694782250/Scion-Masks-of-the-Mythos-Final-Download
42. Crystalpunk Campaign Guide - pdfcoffee.com https://pdfcoffee.com/crystalpunk-campaign-guide-pdf-free.html
43. Experts for Nodejs-Mobile-React-Native Plugins Readme - Linknovate https://www.linknovate.com/search/?query=nodejs-mobile-react-native%20plugins%20readme
44. A Review of TRiSM Frameworks in Artificial Intelligence Systems ... https://onlinelibrary.wiley.com/doi/10.1111/exsy.70213
45. m1rr0rs/gitlab - Gitee https://gitee.com/m1rr0rs/gitlab/blob/master/CHANGELOG.md
46. [PDF] Open Source Used In Firepower 7.4.2 - Cisco https://www.cisco.com/c/dam/en_us/about/doing_business/open_source/docs/Firepower-742-1724171606.pdf
47. Mikhailo Klymash Mykola Beshley Andriy Luntovskyy Editors https://link.springer.com/content/pdf/10.1007/978-3-030-92435-5.pdf
48. DeepSeek AI: Enterprise Implementation Guide | PDF - Scribd https://www.scribd.com/document/917558009/DeepSeek-AI-a-Comprehensive-Guide-for-Enterprise-Implementation
49. MCP Notify | MCP Servers - LobeHub https://lobehub.com/zh/mcp/nirholas-mcp-notify
50. https://mirrors.aliyun.com/debian-ports/indices/ov... https://mirrors.aliyun.com/debian-ports/indices/override.unstable.main
51. PO files — Packages not i18n-ed - Debian https://www.debian.org/international//l10n/po/todo.en.html
52. @nirholas/github-to-mcp - npm https://www.npmjs.com/package/@nirholas/github-to-mcp